

UNIVERSITÀ DI PISA

Decentralised Lending Service

Marco Pernisco — Student ID: 683674

P2P Systems and Blockchains Lab 2025/26 — Trace 1.1

1. Architecture and implementation choices

An ERC-1967 `FundingPoolProxy` stores state and ETH and delegates to the UUPS `FundingPool` implementation. The proxy initializes atomically; implementation initializers are disabled and only the owner may upgrade. The factory deploys a dedicated `LoanContract` for each approved proposal. The pool maintains separate accounting for disposable capital, locked positions, claimable interest and the compensation reserve. The reserve is not loanable capital and contributes no voting power.

A centralized `BitcoinOracle` stores satoshi balances. Its owner publishes values computed by a Python UTXO index over a historical Bitcoin snapshot. Borrowers request updates with a fee; the pool checks the stored liquidity at resolution using 1 BTC = 30 ETH. As required by the trace, Bitcoin ownership is not proved and BTC are not locked.

Voting. Deposits must exceed 100,000 wei. Contributors may vote even when all capital is locked. At resolution, weights are recomputed from current disposable balances; abstentions count as NO and ties reject. Twelve blocks is interpreted as the *minimum* before resolution: voting remains open until the original borrower resolves. The existing ABI name `votingDeadlineBlock` denotes that earliest resolution block. Insufficient pool/BTC liquidity closes the proposal as rejected. Capital is locked proportionally across all positions; division remainders reduce the principal actually disbursed.

Installments. For principal P, rate r, interest $I = \lfloor Pr/100 \rfloor$ and cumulative accepted repayment $C \leq P + I$, define

$$B(C) = \left\lfloor \frac{CP}{P+I} \right\rfloor, \qquad J(C) = C - B(C).$$

Each installment applies the differences from the previous B, J. This interprets the trace's base/interest split for every installment, instead of paying all principal first. Collateral uses differences of the cumulative $\lfloor Jc/100 \rfloor$ target; splitting payments therefore preserves these rounding totals. Excess beyond the entire debt goes to compensation.

Principal refunds prioritize the largest *initial* backing positions, breaking ties by descending numerical address. Merge sort computes this order once per loan; old loans without a cached order are prepared lazily. Gains are credited separately and proportionally to original shares, with distribution dust going to compensation. A pull-payment function transfers each contributor's gain.

Default and termination. Each loan snapshots collateral c at creation. Global collateral starts at 50, increases/decreases by five points for each failed/successful loan and stays within 1–100. Repayment in the due block is timely; later repayment records irreversible default. A valid compensation request records default even if the reserve is empty: a zero payment preserves the claim for a later refill. Compensation forfeits the same amount of future principal recovery; original interest weights remain. The owner may terminate only without active loans; recovery operations remain available.

2. Gas evaluation and testing

Tests use Hardhat 3.18.1, TypeScript 6.0.3, `node:test` and viem 2.57.3, matching the laboratory's testing approach. Solidity 0.8.24 is compiled with optimizer 200 runs and EVM target Berlin. The simulated network also uses Berlin and chain ID 202526, matching the provided genesis.

The following figures are `gasUsed` from actual transaction receipts, including intrinsic gas. The reproducible scenario uses one contributor and separate partial/final repayments. They are measurements of these inputs and storage states, not upper bounds for every possible call.

Operation / scenario	Gas units
`contribute_new`	140,923
`withdraw_partial`	47,933
`requestUpdate`	25,193
`update_new_address`	49,426
`update_existing_address`	32,326
`createLoanProposal`	234,798
`voteOnProposal`	106,936
`executeApprovedLoan_approved`	1,049,458
`repay_partial`	168,082
`repay_final`	90,716
`claimInterestGain`	24,691
`markDefaulted`	103,758
`requestCompensation_paid`	54,841
`requestCompensation_empty`	45,975
`repay_late`	138,782
`executeApprovedLoan_rejected`	90,860
`withdrawFees`	30,868
`upgradeToAndCall`	37,769
`terminate`	36,295
`withdraw_final`	32,412

The oracle fee is calibrated to a first nonzero update of a 34-character historical address: $49,426 \times 0.1$ gwei $= 4,942,600,000,000$ wei. Existing-slot updates are cheaper. Changing the compiler, bytecode or scenario requires recalibration. Public getters used with `eth_call` do not incur a paid transaction fee.

TypeScript scenarios cover current-balance voting, abstentions/ties, zero-weight votes, proportional allocation, rounding, installments, irreversible defaults, repeated compensation, termination, oracle access, UUPS upgrades and reentrancy. Native Solidity tests include fuzzed repayment conservation and cumulative rounding. Python tests exercise genesis exclusion, malformed/incomplete data, atomic indexing, persistent events, retries, receipt failures and the demo's fresh-balance checks. Coverage and Hardhat gas statistics are generated with `npm run test:coverage` and `npm run test:gas`. Coverage instrumentation does not overwrite the ordinary gas measurements.

3. Reentrancy modification and malicious strategies

`FundingPoolVulnerable` inherits the actual pool and changes only the withdrawal implementation for the test. It caches the position and disposable total, transfers ETH first, and then writes stale balances:

```
uint256 beforePosition = totalContributed[msg.sender];
uint256 beforePool = totalUnlockedFunds;
(bool sent, ) = payable(msg.sender).call{value: amount}("");
if (!sent) revert TransferFailed();
totalContributed[msg.sender] = beforePosition - amount;
totalUnlockedFunds = beforePool - amount;
```

The attacker contributes once and recursively calls `withdraw(amount)` from its receive function. Cached assignments leave the same claim available during reentry. Merely moving a checked live subtraction after the transfer could instead underflow and revert the attack in Solidity 0.8; the tested variant demonstrates an actual exploitable modification.

The same attacker targets both proxy deployments. It drains others' deposits only from the vulnerable variant. The normal pool updates accounting before the external call; recursive withdrawals fail and the original legitimate withdrawal can finish. The vulnerable implementation is never used by the setup script. Checks-effects-interactions and tests address this scenario without claiming a general proof against every possible interaction.

Unfair remuneration without reentrancy. A whale contributing 6 ETH can outvote honest contributors holding 4 ETH, approve a 10 ETH loan to a colluding borrower address and deliberately default. Because any sufficiently funded Bitcoin address can be cited, the didactic liquidity check does not prevent this. The attacker moves 6 ETH of its own capital but captures the minority's 4 ETH; the net gain is not the full 10 ETH. Priority refunds also favor large initial positions: with backing 6/4, a base repayment of 6 restores the whale's position before the smaller contributor recovers anything. These are consequences of the specified economic rules, not reasons to silently replace them.

Operational limitations. The oracle rejects incomplete snapshots and excludes the unspendable genesis reward. It applies blocks in order and validates their payload/Merkle integrity, while trusting the supplied historical data rather than implementing full Bitcoin consensus. The replacement seven-file dataset contains 162,844 connected blocks. Offline replay successfully validated the required first 131,000 blocks through height 130,999, reconstructing 299,208 UTXOs and 153,986 address entries. The earlier incomplete-data issue is resolved. The complete Python workflow also passed on Geth 1.13.14 with the unchanged course genesis and 10-second Clique blocks: 15 successful transaction receipts, 20 final state checks and both event queues completed. A 1 ETH loan at 5% was repaid in two 0.525 ETH installments; the contributor claimed 0.025 ETH gross gain and compensation retained 0.025 ETH. Evidence is recorded in `docs/geth-demo.json`. No balance is published after a failed sync.

The Python services durably enqueue events before advancing the cursor. They resume from deployment, retry temporary errors and recover the same signed transaction after a timeout. Checkpoints detect chain replacement; nonce conflicts are retained for operator inspection. This model targets the single-node private chain, without automatic reorganization recovery.

Removing empty positions and caching sorted refund order reduces needless work. Operations still grow with active contributor count and must fit the block gas limit. A compatible V2 test verifies ownership and storage preservation; existing independent loan contracts keep their original bytecode after a pool upgrade.

4. Brief user manual

Use supported Node 22.13.0+ (supported even-numbered major), Python 3.10+, and the pinned Geth 1.13.14 Clique implementation. Run from the project root:

```
./setup.sh
npm run typecheck
npm test
./.venv/bin/python -m unittest discover -s python/tests -v
npm run test:gas
```

Setup installs locked npm dependencies, creates the Python environment, compiles and obtains Geth. Validate complete decoded mainnet block files before deployment:

```
./.venv/bin/python python/oracle/check_data.py \
  --data-dir bitcoin --summary
```

Continue only when the JSON reports `valid: true`, 131,000 blocks and height 130,999. The summary supplies example addresses and their historical liquidity. The checker requires no private keys or Ethereum connection.

In terminal 1 initialize and start the chain:

```
cd private-chain
./geth --datadir=./data init project2526genesis.json
./geth --datadir=./data --keystore=./keystore \
  --networkid=202526 --http --http.addr=127.0.0.1 \
  --http.port=8545 --http.api=eth,net,web3 --mine \
  --miner.gasprice=100000000 \
  --miner.etherbase=d278d247a52c550508ea2b2c9321d816238fb523 \
  --unlock d278d247a52c550508ea2b2c9321d816238fb523 \
  --password password.txt --allow-insecure-unlock --ipcdisable
```

From the project root, terminal 2 deploys:

```
./.venv/bin/python python/setup.py
```

Python setup first rebuilds ordinary bytecode, including after coverage runs. The provided prefunded account only transfers ETH to new accounts. The new deployer deploys the oracle, implementation and initialized proxy. Setup validates chain ID, receipts, code and initialization and writes the service environments including the deployment block. Running setup again creates a separate deployment.

Start the services in separate terminals:

```
./.venv/bin/python python/oracle/oracle_service.py
./.venv/bin/python python/voter/auto_voter.py
```

Then choose a funded address from the successful snapshot summary:

```
./.venv/bin/python python/simulate.py \
  --btc-address YOUR_VALIDATED_ADDRESS --principal 1
```

The demo requests a fresh balance, verifies sufficient liquidity, contributes, proposes, waits for the configured YES vote, resolves and repays in two installments before claiming gain. It checks receipts and prints balances/state. Use a fresh deployment for the revised loan rules; no previous chain is modified by updating repository files.

Keep the services' separate SQLite databases when restarting after an RPC error. The README describes recovery, dataset requirements, report regeneration and submission packaging. The current workflow has passed on an isolated Geth instance with the course genesis and valid Bitcoin data; prepare the final course ZIP with the verified sources and report.